

03.3: Docker Compose



UMD DATA605 - Big Data Systems

03.3: Docker Compose

Instructor: Dr. GP Saggese, gsaggese@umd.edu

1 / 8

2 / 8: Docker Compose

Docker Compose

- **Manage multi-container apps running on a single node**
 - Describe app in a single *declarative* YAML file
 - Avoid long Docker command scripts
 - Compose interacts with Docker API to achieve the requested state
 - Examples:
 - Client app and Postgres DB
 - Microservices: Web front-end, Ordering, Back-end DB
 - In 2020, Docker Compose became an open standard for “code-to-cloud” process
- **Manage multi-container apps running on multiple hosts**
 - Docker Stacks / Swarm
 - Kubernetes

2 / 8

- **Docker Compose**
 - *Manage multi-container apps running on a single node*
 - Docker Compose is a tool that helps you manage applications that consist of multiple containers on a single machine. Instead of running each container with a long list of commands, you can use a single YAML file to describe the entire application setup. This makes it easier to manage and understand.
 - The YAML file is *declarative*, meaning you specify what you want, and Docker Compose takes care of making it happen by interacting with the Docker API. This simplifies the process and reduces the chance of errors.
 - Some common examples of applications you might manage with Docker Compose include a client application that connects to a Postgres database or a set of microservices like a web front-end, an ordering service, and a back-end database.
 - In 2020, Docker Compose became an open standard, which means it is widely accepted and used for moving code from development to production environments, often referred to as the “code-to-cloud” process.
 - *Manage multi-container apps running on multiple hosts*
 - When you need to manage applications that run across multiple machines, you can use tools like Docker Stacks or Docker Swarm. These tools help coordinate and manage containers across different hosts.
 - Kubernetes is another popular tool for managing multi-container applications across multiple hosts. It provides advanced features for scaling, managing, and deploying applications in a distributed environment.

3 / 8: Docker Compose: Tutorial Example

Docker Compose: Tutorial Example

- Default name for a Compose file is `docker-compose.yml`
 - Specify `-f` for custom filenames
- **Top-level keys:**
 - `version:`
 - Mandatory first line for API version
 - Use latest version, typically 3 or higher
 - `services:`
 - Define microservices
 - `networks:`
 - Create new networks
 - Default `bridge` network connects containers on the same Docker host
 - `volumes:`
 - Create new volumes
- **Key in services** describes a “service” in terms of container

```
version: "3.8"
services:
  web-fe:
    build: .
    command: python app.py
    ports:
      - target: 5000
        published: 5001
    networks:
      - counter-net
    volumes:
      - type: volume
        source: counter-vol
        target: /code
  redis:
    image: "redis:alpine"
    networks:
      counter-net:
networks:
  counter-net:
volumes:
  counter-vol:
```

3 / 8

- **Default name for a Compose file is `docker-compose.yml`**
 - This is the standard file name Docker Compose looks for by default. If your file has a different name, you need to specify it using the `-f` option when running Docker Compose commands.
- **Top-level keys:**
 - **`version:`**
 - This key specifies the version of the Docker Compose file format. It’s important to use the latest version, usually 3 or higher, to ensure compatibility with the latest features.
 - **`services:`**
 - This section is where you define the different microservices that make up your application. Each service corresponds to a container.
 - **`networks:`**
 - Networks allow containers to communicate with each other. By default, Docker uses a `bridge` network, which connects containers on the same host.
 - **`volumes:`**
 - Volumes are used to persist data generated by and used by Docker containers. They are especially useful for keeping data even when containers are removed.
- **Key in services** describes a “service” in terms of a container
 - **Inner keys** specify parameters that are similar to those used in the `docker run` command. These include settings like the image to use, commands to run, ports to expose, and more.

The YAML example on the right illustrates a simple setup with two services: a web frontend (`web-fe`) and a Redis database. The `web-fe` service is built from the current directory, runs a

Python application, and exposes port 5001. It also uses a custom network (`counter-net`) and a volume (`counter-vol`) to store code. The `redis` service uses a lightweight Redis image and connects to the same network, allowing it to communicate with the `web-fe` service.

4 / 8: Docker Compose: Commands

Docker Compose: Commands

```
> docker compose --help
Usage: docker compose [OPTIONS] COMMAND
Options:
  --env-file string      Specify an alternate environment file.
  -f, --file stringArray Compose configuration files
  -p, --project-name string Project name

Commands:
  build      Build or rebuild services
  convert    Converts the compose file to platform's canonical format
  cp         Copy files/folders between a service container and the local filesystem
  create     Creates containers for a service.
  down       Stop and remove containers, networks
  events     Receive real time events from containers.
  exec       Execute a command in a running container.
  images     List images used by the created containers
  kill       Force stop service containers.
  logs       View output from containers
  ls         List running compose projects
  pause     Pause services
  port       Print the public port for a port binding.
  ps        List containers
  pull      Pull service images
  push      Push service images
  restart   Restart containers
  rm        Removes stopped service containers
  run       Run a one-off command on a service.
  start     Start services
  stop      Stop services
  top       Display the running processes
  unpause   Unpause services
  up        Create and start containers
  version   Show the Docker Compose version information
```

4 / 8

- **Docker Compose Overview:** Docker Compose is a tool used for defining and running multi-container Docker applications. It allows you to manage the lifecycle of your application services using a simple configuration file.
- **Usage and Options:**
 - *Usage:* The command `docker compose [OPTIONS] COMMAND` is the basic structure for using Docker Compose. You specify options and a command to perform specific tasks.
 - **–env-file:** This option lets you specify an alternate environment file, which is useful for managing different configurations for different environments (e.g., development, testing, production).
 - **–f, –file:** Allows you to specify one or more Compose configuration files. This is helpful when you have multiple configuration files for different setups.
 - **–p, –project-name:** Sets a custom project name, which is useful for organizing and managing multiple projects.
- **Common Commands:**
 - **build:** Builds or rebuilds services defined in the Compose file. This is often the first step to ensure all services are up-to-date.
 - **up:** Creates and starts containers as defined in the Compose file. This is the primary command to launch your application.

- **down:** Stops and removes containers, networks, and other resources created by up. Use this to clean up your environment.
- **logs:** Displays the output from containers, which is crucial for debugging and monitoring.
- **ps:** Lists all containers managed by the current Compose project, providing a quick overview of what's running.
- **exec:** Executes a command inside a running container, useful for debugging or running maintenance tasks.
- **restart:** Restarts containers, which can be necessary after configuration changes or updates.
- **version:** Displays the version information of Docker Compose, ensuring compatibility and troubleshooting.
- **Context and Importance:**
 - Docker Compose simplifies the management of complex applications by allowing you to define services, networks, and volumes in a single file.
 - It is particularly useful in development and testing environments where you need to quickly spin up and tear down services.
 - Understanding these commands and options is crucial for effectively using Docker Compose to manage containerized applications.

5 / 8: Docker Compose: Commands

Docker Compose: Commands

- Build the containers for the services
 - > `docker compose build`
- Pull the needed images for the services
 - > `docker compose pull`
- Show the running services
 - > `docker compose ps`
- Show the status of the service
 - > `docker compose ls`
- Bring up the entire service
 - > `docker compose up`
- Rebuild after trying out some changes in dockerfile/compose file
 - > `docker-compose up --build --force-recreate`

5 / 8

- **Build the containers for the services**
 - This command, `docker compose build`, is used to create Docker images based on the instructions provided in the Dockerfile. It compiles the necessary components and dependencies into a container image. This is the first step in setting up your services,

ensuring that everything is packaged correctly before running.

- **Pull the needed images for the services**
 - The `docker compose pull` command fetches the latest versions of the images specified in your Docker Compose file from a container registry. This is useful when you want to ensure that you have the most up-to-date images before starting your services.
- **Show the running services**
 - By using `docker compose ps`, you can view a list of all the services that are currently running. This command provides a quick overview of the status and health of your containers, helping you monitor your application's components.
- **Show the status of the service**
 - The `docker compose ls` command lists all the Docker Compose projects on your system, showing their current status. This is helpful for managing multiple projects and understanding which ones are active or need attention.
- **Bring up the entire service**
 - With `docker compose up`, you can start all the services defined in your Docker Compose file. This command initializes and runs the containers, making your application available for use. It's a straightforward way to launch your entire stack with a single command.
- **Rebuild after trying out some changes in dockerfile/compose file**
 - The command `docker-compose up --build --force-recreate` is used when you have made changes to your Dockerfile or Compose file and need to apply those changes. It rebuilds the images and recreates the containers, ensuring that your updates are reflected in the running services. This is crucial for testing and deploying new features or fixes.

6 / 8: Docker Compose: Commands

Docker Compose: Commands

- Show the processes inside each container

```
> docker compose top
counter_app-redis-1
UID    PID    PPID   C    STIME  TTY   TIME      CMD
999    49590  49549  0    10:40  ?     00:00:02  redis-server *:6379

counter_app-web-fe-1
UID    PID    PPID   C    STIME  TTY   TIME      CMD
root   49614  49574  0    10:40  ?     00:00:00  python app.py
root   49734  49614  1    10:40  ?     00:00:08  /usr/local/bin/python /
```

6 / 8

- **Show the processes inside each container:** This command is useful when you want

to see what processes are running inside your Docker containers. It provides a snapshot of the current state of each container's processes, similar to the `top` command in Linux, but specifically for Docker containers.

- **Command:** `docker compose top` is the command used to display the processes. This command is part of Docker Compose, a tool for defining and running multi-container Docker applications. It helps you manage and monitor the processes within your containers.
- **Example Output:** The output shows two containers, `counter_app-redis-1` and `counter_app-web-fe-1`. For each container, it lists details like **UID** (User ID), **PID** (Process ID), **PPID** (Parent Process ID), **C** (CPU usage), **STIME** (Start Time), **TTY** (Terminal Type), **TIME** (CPU Time), and **CMD** (Command being executed).
- **Understanding the Output:**
 - For `counter_app-redis-1`, the process is `redis-server *:6379`, indicating that a Redis server is running.
 - For `counter_app-web-fe-1`, there are two processes: `python app.py` and `/usr/local/bin/python /code/app.py`, showing that a Python application is running, likely a web application.
- **Importance:** This command is particularly useful for debugging and monitoring purposes. It allows you to verify that the expected processes are running and helps in diagnosing issues related to resource usage or unexpected behavior within your containers.

7 / 8: Docker Compose: Commands

Docker Compose: Commands

- Build the containers for the services

```
> docker compose down
[+] Running 3/2
Container counter_app-redis-1   Removed
Container counter_app-web-fe-1  Removed
Network counter_app_counter-net Removed
```

- Shutdown service removing the volume (i.e., resetting state)

```
> docker-compose down -v
```

- Shutdown service removing images and volume

```
> docker-compose down -v --rmi all
```

7 / 8

- **Build the containers for the services**
 - This command is used to stop and remove all the containers, networks, and other resources created by Docker Compose.

- The example shows the command `docker compose down`, which is used to clean up the environment by removing containers and networks associated with the services.
- The output indicates that two containers (`counter_app-redis-1` and `counter_app-web-fe-1`) and one network (`counter_app_counter-net`) have been successfully removed.
- This is useful when you want to start fresh or make changes to the configuration without leftover data from previous runs.
- **Shutdown service removing the volume (i.e., resetting state)**
 - The command `docker-compose down -v` is used to stop the services and remove the associated volumes.
 - Removing volumes is important when you want to reset the state of your application, as volumes store persistent data.
 - This command ensures that any data stored in volumes is deleted, allowing you to start with a clean slate the next time you bring up the services.
- **Shutdown service removing images and volume**
 - The command `docker-compose down -v --rmi all` goes a step further by also removing all images used by the services.
 - This is useful when you want to completely clean up your environment, including the images, which can help free up disk space.
 - Removing images means that the next time you start the services, Docker will need to rebuild or pull the images again, which can be useful if you want to ensure you are using the latest versions.

8 / 8: Docker Compose: Tutorial

Docker Compose: Tutorial

- Example taken from <https://github.com/nigelpoulton/counter-app>
- `tutorial_docker_compose`
 - > `cd tutorials/tutorial_docker_compose`
 - > `vi tutorial_docker_compose.md`

8 / 8

- **Docker Compose: Tutorial**
 - Example taken from <https://github.com/nigelpoulton/counter-app>

- This bullet point indicates that the tutorial is based on a practical example from a GitHub repository. The repository, created by Nigel Poulton, contains a sample application called “counter-app.” This app is likely used to demonstrate how Docker Compose can be utilized to manage multi-container Docker applications.
- **tutorial_docker_compose**
 - This link directs you to another GitHub repository where the tutorial files are stored. It is part of a series of tutorials for a class, possibly related to data science or software development, given the context of Docker Compose usage. This repository likely contains additional resources or instructions to help you follow along with the tutorial.
- **Command Line Instructions**
 - ```
> cd tutorials/tutorial_docker_compose
```

```
> vi tutorial_docker_compose.md
```
  - These commands are meant to guide you through accessing and editing the tutorial files. The first command, `cd`, changes the directory to where the tutorial files are located. The second command, `vi`, opens a text editor to view or edit the `tutorial_docker_compose.md` file. This file likely contains detailed instructions or explanations related to the Docker Compose tutorial.